Unit 2 Assignment 3

Eigenvalues & Eigenvectors — Group Report

Group members: Razan, Lamia, Sara, Ahmed

This report summarizes our manual calculations for the six missions and the NumPy results we used to verify them. All missions use the matrix below, and every manual answer matched the NumPy output.

$$A = \begin{vmatrix} 2 & 1 \\ 1 & 2 \end{vmatrix}$$

Mission 1 — Discovering Eigenvectors

Vector	Manual: A · v	Result	Direction
$v_1 = [1, 1]$	[2(1)+1(1), 1(1)+2(1)]	$[3, 3] = 3 \cdot v_1$	Same direction, stretched ×3
$v_2 = [1, -1]$	[2(1)+1(−1), 1(1)+2(−1)]	$[1, -1] = 1 \cdot v_2$	Same direction, same length
u = [1, 0] (comparison)	[2(1)+1(0), 1(1)+2(0)]	[2, 1]	Direction changed

NumPy: `A @ v1 → [3, 3]`, `A @ v2 → [1, −1]` ✓ **verified**

Both v_1 and v_2 remain on the same direction. They are called **eigenvectors** because the matrix only scales them ($A \cdot v = \lambda \cdot v$) without changing their direction.

> **AI Insight:** A matrix stretches space by different amounts in different directions, so most vectors are pushed toward a new direction. Only a few special directions, the eigenvectors, are stretched along themselves, so they stay on the same line.

Mission 2 — Finding Eigenvalues

Characteristic equation: $\det(A - \lambda I) = 0$

$$A - \lambda I = \begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix}$$

- $\det(A - \lambda I) = (2 - \lambda)(2 - \lambda) - (1)(1) = 0$
- $\lambda^2 - 4\lambda + 4 - 1 = 0 \rightarrow \mathbf{\lambda^2 - 4\lambda + 3 = 0}$
- $(\lambda - 3)(\lambda - 1) = 0 \rightarrow \mathbf{\lambda_1 = 3, \lambda_2 = 1}$

NumPy: `np.linalg.eigvals(A) → [3, 1]` ✓ **verified**

> **AI Insight:** An eigenvalue tells how much its eigenvector is stretched or shrunk by the transformation, and whether it is flipped when negative. Here the direction [1, 1] is stretched 3 times and the direction [1, −1] keeps its size.

Mission 3 — Finding Eigenvectors

Solve $(A - \lambda I) \cdot v = 0$ for each eigenvalue:

λ	A − λI	Equation	Eigenvector	Normalized (÷√2)
3	[−1 1; 1 −1]	−x + y = 0 → y = x	[1, 1]	[0.707, 0.707]
1	[1 1; 1 1]	x + y = 0 → y = −x	[1, −1]	[0.707, −0.707]

NumPy `np.linalg.eig(A)` returns the eigenvectors as columns: [0.707, 0.707] and [−0.707, 0.707] ✓ **verified**
The second one is −1 × [0.707, −0.707]. It is the same eigenvector, because multiplying an eigenvector by any non-zero number, including −1, keeps it on the same line.

> **AI Insight:** Each eigenvalue is a different stretching amount, and each stretching happens along its own direction. Solving $(A - \lambda I) \cdot v = 0$ gives a different solution for each λ, so every eigenvalue has its own eigenvector.

Mission 4 — Geometric Interpretation

From $A \cdot v = \lambda \cdot v$, using v = [1, 1] as an example:

Eigenvalue	Result	Interpretation
λ = 3	[3, 3]	Stretched **3 times longer**, same direction
λ = 0.5	[0.5, 0.5]	Shrunk to **half** its length, same direction
λ = −2	[−2, −2]	Stretched **2 times** and **flipped** to the opposite direction

> **AI Insight:** The **size** of λ controls scaling: $|\lambda| > 1$ stretches, $|\lambda| < 1$ shrinks, and $|\lambda| = 1$ keeps the length. The **sign** of λ controls direction: positive keeps it, negative flips it.

Mission 5 — Matrix Diagonalization

P (eigenvectors as columns) = $\begin{vmatrix} 1 & 1 \\ 1 & -1 \end{vmatrix}$ D (eigenvalues on the diagonal) = $\begin{vmatrix} 3 & 0 \\ 0 & 1 \end{vmatrix}$

$\det(P) = (1)(-1) - (1)(1) = -2 \rightarrow P^{-1} = (1/-2) \cdot [-1\ -1; -1\ 1] =$ $\begin{vmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{vmatrix}$

NumPy: `P @ D @ P_inv → [[2, 1], [1, 2]] = A` ✓ **verified: $A = P D P^{-1}$**

Why diagonal matrices are easier: all entries off the diagonal are zero, so multiplying by D just scales each coordinate separately. Powers are easy (D^n raises each diagonal number to the power n, so $A^n = P D^n P^{-1}$). The determinant is the product of the diagonal, and the inverse is 1 divided by each diagonal number.

> **AI Insight:** Diagonalization breaks a complex transformation into simple, independent scalings along the eigenvector directions. In Machine Learning it is the basis of PCA, it makes repeated matrix calculations fast, and it helps analyze how algorithms behave over many steps.

Mission 6 — Eigenvalues in AI

1. **Why is PCA used?** To reduce the number of features while keeping most of the important information. This makes models faster, reduces noise and overfitting, and lets us visualize data in 2D or 3D.
2. **Why are eigenvalues important in PCA?** PCA takes the eigenvectors of the data's covariance matrix as new directions (principal components). Each eigenvalue tells how much variance, or information, lies along its direction.
3. **Why sort eigenvalues from largest to smallest?** To rank the directions by importance, so the components with the most information are picked first.
4. **Which eigenvectors should be selected?** The ones with the largest eigenvalues (the top k), usually enough to keep about 90–95% of the total variance.
5. **What if we remove the eigenvector with the largest eigenvalue?** We would lose the most important information in the data, the direction with the most variance, and the model would perform much worse.

Example: if A were a covariance matrix, its sorted eigenvalues 3 and 1 hold 3/4 = **75%** and 1/4 = **25%** of the information. Keeping only the first direction reduces 2 features to 1 while keeping 75% of the variance. ✓ **NumPy**

> **AI Insight:** Eigenvalues measure how much information each direction holds. By keeping only the eigenvectors with the largest eigenvalues and dropping those with small eigenvalues, which hold little information and mostly noise, we reduce the number of dimensions while preserving most of the useful information.

Conclusion

We found that A = [[2, 1], [1, 2]] has eigenvalues 3 and 1 with eigenvectors [1, 1] and [1, −1], confirmed that the matrix only scales these vectors, interpreted how eigenvalues stretch, shrink and flip vectors, diagonalized A as $P\,D\,P^{-1}$, and explained how sorting eigenvalues lets PCA reduce dimensions while keeping the most information. All manual calculations were verified with NumPy and matched.